HOME ASSIGNMENT - 3
ID : 12341550

TASK1

FILE : proc.h

under struct proc

...

```
  int priorityMark;
```

TASK2

FILE : proc.c

under alloproc():

…

```
p->priorityMark = 0;
```

modified sheduler function:

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;
  for (;;)
  {
    sti();
    struct proc *highest_priority_p = 0;
    int highest_priority = 1000;
    acquire(&ptable.lock);

    for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    {
```

```
      if (p->state == RUNNABLE)
      {
        if (p->priorityMark < highest_priority)
        {
          highest_priority_p = p;
        }
      }
    }
    if (highest_priority_p != 0)
    {
      p = highest_priority_p;
      c->proc = p;
      switchuvm(p);
      p->state = RUNNING;

      p->shed_count++;

      swtch(&(c->scheduler), p->context);
      c->proc = 0;
    }
    release(&ptable.lock);
  }
}
```

FILE : sysproc.c

```
{
  int priority;
  if (argint(0, &priority) < 0)
  {
    return -1;
  }
  // Assign the given priority value to the calling process 's
priority field
```

```
  struct proc * p = myproc();
  p->priorityMark = priority;
  return 0;
}
```

TASK3

FILE : syscall.h

```
#define SYS_setpriority 30
```

FILE : syscall.c

```
extern int sys_setpriority(void);

under syscalls[]
…
[SYS_setpriority] sys_setpriority,
```

TASK4

FILE : user.h

```
int setpriority(int);
```

FILE : usys.S

```
SYSCALL(setpriority)
```

TASK5

FILE : prioritytest.c (new)

```
#include "types.h"
#include "stat.h"
#include "user.h"
int main(int argc, char *argv[])
{
   int pid1, pid2, pid3, pid4, pid5;
   printf(1, " Starting priority scheduling test ...\n ");

   pid1 = fork();
   if (pid1 == 0)
   {
      setpriority(10);
      printf(1, " Child 1 (pid %d) with priority (10) started .\n
",
            getpid());
      for (int i = 0; i < 50000000; i++)
      {
      } // Busy loop
      printf(1, " Child 1 finished .\n ");
      exit();
   }

   pid2 = fork();
   if (pid2 == 0)
   {
      setpriority(50);
      printf(1, " Child 2 (pid %d) with priority (50) started .\n
 ",
            getpid());
      for (int i = 0; i < 50000000; i++)
      {
      } // Busy loop
      printf(1, " Child 2 finished .\n ");
      exit();
   }
```

```
    pid3 = fork();
    if (pid3 == 0)
    {
        setpriority(70); // Low priority
        printf(1, " Child 3 (pid %d) with priority (70) started .\n
",
            getpid());
        for (int i = 0; i < 50000000; i++)
        {
        } // Busy loop
        printf(1, " Child 3 finished .\n ");
        exit();
    }

    pid4 = fork();
    if (pid4 == 0)
    {
        setpriority(30); // Low priority
        printf(1, " Child 4 (pid %d) with priority (30) started .\n
",
            getpid());
        for (int i = 0; i < 50000000; i++)
        {
        } // Busy loop
        printf(1, " Child 4 finished .\n ");
        exit();
    }

    pid5 = fork();
    if (pid5 == 0)
    {
        setpriority(5); // Low priority
        printf(1, " Child 5 (pid %d) with priority (5) started .\n
",
```

```
            getpid());
        for (int i = 0; i < 50000000; i++)
        {
        } // Busy loop
        printf(1, " Child 5 finished .\n ");
        exit();
            }
    wait();
    wait();
    wait();
    wait();
    wait();
    printf(1, " Priority scheduling test complete .\n ");
    exit();
}
```

OUTPUT :

